Assignment – 4

Deep Learning (UCS3008SE3)

B.Tech. CSE AIML (Semester - VI)

UNIT: 4

Date of Submission: April 17, 2026

Class:- 3CSE-4

Submitted By: Sneha Verma

Roll No. – CS-2341409

Submitted To: Dr. Ajay Kumar

Q1. Explain the basic architecture of a Convolutional Neural Network (CNN). Describe the roles of convolutional layers, activation functions, pooling layers, and fully connected layers in image classification tasks.

Answer:

A Convolutional Neural Network (CNN) is a class of deep learning models specifically designed to process structured grid data such as images. The basic architecture of a CNN consists of a sequence of layers, each performing a specific transformation on the input data. The typical pipeline for image classification is:

Input → Convolutional Layer → Activation (ReLU) → Pooling Layer → ... → Fully Connected Layer → Softmax → Output

1. Input Layer

The input layer receives the raw pixel values of an image. For a color image of size 224×224, the input would have dimensions 224 × 224 × 3 (height × width × RGB channels).

2. Convolutional Layers

Convolutional layers are the core building blocks of a CNN. They apply a set of learnable filters (kernels) to the input. Each filter slides across the input spatially, computing the dot product between the filter weights and the local region of the input. This operation produces a feature map (also called activation map) that highlights the presence of specific features such as edges, corners, and textures. Multiple filters are used per layer, each detecting a different feature. Early convolutional layers detect low-level features (edges, gradients), while deeper layers capture high-level features (shapes, objects).

3. Activation Functions

After each convolution operation, a non-linear activation function is applied element-wise to the feature map. The most commonly used activation function is the Rectified Linear Unit (ReLU), defined as $f(x) = \max(0, x)$. ReLU introduces non-linearity into the model, enabling it to learn complex patterns. Without activation functions, stacking multiple convolutional layers would be equivalent to a single linear transformation, severely limiting the model's representational capacity.

4. Pooling Layers

Pooling layers perform spatial down-sampling of feature maps, reducing their height and width while retaining the most important information. The two common types are:

- Max Pooling: Selects the maximum value from each local region (e.g., 2×2 window).
- Average Pooling: Computes the average value from each local region.

Pooling reduces computational cost, the number of parameters, and provides a degree of translation invariance — meaning that small shifts in the input do not significantly change the output.

5. Fully Connected (Dense) Layers

After several convolutional and pooling layers, the feature maps are flattened into a onedimensional vector and fed into one or more fully connected layers. These layers perform the final classification by learning non-linear combinations of the extracted features. The last fully connected layer typically has as many neurons as the number of output classes and uses a Softmax activation function to produce probability scores for each class.

Q2. Compare Convolutional Neural Networks (CNNs) with Multilayer Perceptrons (MLPs). Why are CNNs more suitable for image-related tasks? Discuss concepts such as local connectivity, weight sharing, and parameter efficiency.

Answer:

Although both Multilayer Perceptrons (MLPs) and Convolutional Neural Networks (CNNs) are feed-forward neural networks, they differ fundamentally in how they process input data, particularly images.

Multilayer Perceptrons (MLPs)

An MLP connects every input neuron to every neuron in the next layer (fully connected). For an image of size 224 × 224 × 3 = 150,528 pixels, the first hidden layer of 1,000 neurons alone would require over 150 million parameters. This makes MLPs computationally expensive and highly prone to overfitting when applied to image data. Moreover, MLPs treat each pixel independently, discarding all spatial relationships and neighborhood information.

Why CNNs Are Better for Images

- **Local Connectivity:** Instead of connecting every neuron to all input pixels, each neuron in a convolutional layer is connected only to a small local region (receptive field) of the input. This preserves spatial locality and captures patterns like edges and textures effectively.
- **Weight Sharing:** The same filter (set of weights) is applied across the entire input image. This means the network can detect a feature (e.g., a vertical edge) regardless of its position in the image. Weight sharing drastically reduces the number of learnable parameters.
- **Parameter Efficiency:** Due to local connectivity and weight sharing, CNNs have significantly fewer parameters than an equivalent MLP. For example, a convolutional layer with 32 filters of size 3×3 on a 3-channel input has only 32 × (3×3×3 + 1) = 896 parameters, compared to millions in an MLP.

- **Spatial Hierarchy:** CNNs inherently capture spatial hierarchies — lower layers learn edges, middle layers learn textures and parts, and deeper layers learn entire objects. MLPs cannot learn such hierarchies from raw pixels.

Comparison Table

Feature	MLP	CNN
Connectivity	Fully connected	Locally connected
Parameter Count	Very high	Significantly lower
Weight Sharing	No	Yes
Spatial Awareness	None	High (preserves structure)
Translation Invariance	None	Yes (via pooling)
Suitability for Images	Poor	Excellent
Overfitting Risk	High (more params)	Lower (fewer params)

In summary, CNNs exploit the spatial structure of images through local connectivity, weight sharing, and hierarchical feature extraction, making them far more suitable and efficient for image-related tasks than MLPs.

Q3. Explain the working of convolution operation in CNNs with a neat diagram. Define kernel, stride, padding, and feature maps, and discuss how they influence the output size.

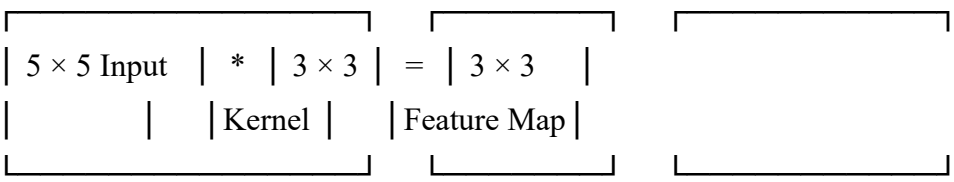
Answer:

Convolution Operation

The convolution operation is the fundamental operation in CNNs. It involves sliding a small matrix called a kernel (or filter) over the input data. At each position, the kernel performs an element-wise multiplication with the overlapping input region, and the results are summed to produce a single value in the output. This output is called a feature map.

For example, consider a 5×5 input matrix and a 3×3 kernel. The kernel starts at the top-left corner, computes the dot product with the 3×3 region, writes the result, then slides to the right. This process continues row by row until the entire input is covered.

Diagram Illustration:



The kernel slides over the input with a given stride, computing dot products at each position.

Key Definitions

- **Kernel:** A kernel (or filter) is a small learnable weight matrix (e.g., 3×3, 5×5) that is convolved with the input to extract features. Different kernels detect different features such as horizontal edges, vertical edges, or corners.
- **Stride:** Stride is the number of pixels by which the kernel moves after each operation. A stride of 1 means the kernel moves one pixel at a time, producing a larger output. A stride of 2 means it moves two pixels at a time, reducing the output size.
- **Padding:** Padding involves adding extra pixels (usually zeros) around the border of the input. "Valid" padding means no padding (output shrinks). "Same" padding adds enough zeros so that the output has the same spatial dimensions as the input.
- **Feature Map:** A feature map is the output produced after applying a single kernel to the input. Each kernel produces one feature map. If we apply 32 kernels, we get 32 feature maps, forming the output volume.

Output Size Formula

The output size of a convolutional layer is calculated as:

$$O = (W - K + 2P) / S + 1$$

Where:

W = Input size (width or height)

K = Kernel size

P = Padding

S = Stride

Example: For $W = 5$, $K = 3$, $P = 0$, $S = 1$:

$O = (5 - 3 + 0) / 1 + 1 = 3$

So a 5×5 input with a 3×3 kernel, no padding, and stride 1 produces a 3×3 feature map.

Q4. Describe the architecture of LeNet-5. Explain its layers, activation functions, and significance in the history of deep learning. Why is LeNet considered a pioneering CNN architecture?

Answer:

LeNet-5 was proposed by Yann LeCun and his colleagues in 1998. It was one of the earliest convolutional neural networks and was specifically designed for handwritten digit recognition, achieving remarkable success on the MNIST dataset.

Architecture of LeNet-5

LeNet-5 consists of 7 layers (excluding the input), with the following structure:

- **Input Layer:** Accepts a grayscale image of size 32 × 32 pixels.
- **C1 – Convolutional Layer:** 6 convolutional filters of size 5×5 with stride 1 and no padding. Output: 6 feature maps of size 28×28. Activation: Tanh (or Sigmoid).
- **S2 – Subsampling (Pooling) Layer:** Average pooling with 2×2 filter and stride 2. Output: 6 feature maps of size 14×14. This layer reduces spatial dimensions by half.
- **C3 – Convolutional Layer:** 16 convolutional filters of size 5×5. Output: 16 feature maps of size 10×10. Activation: Tanh.
- **S4 – Subsampling (Pooling) Layer:** Average pooling with 2×2 filter and stride 2. Output: 16 feature maps of size 5×5.
- **C5 – Fully Connected Convolutional Layer:** Fully connected convolutional layer with 120 filters of size 5×5. Since input is 5×5, this produces 120 feature maps of size 1×1. Activation: Tanh.
- **F6 – Fully Connected Layer:** Fully connected layer with 84 neurons. Activation: Tanh.
- **Output Layer:** 10 neurons corresponding to digits 0–9. Uses Euclidean Radial Basis Function (RBF) for classification.

Significance and Pioneering Contributions

- LeNet-5 was the first CNN to demonstrate that deep networks with backpropagation could be successfully trained for real-world applications.
- It introduced the classic CNN pipeline: Convolution → Pooling → Fully Connected, which remains the foundation of modern CNNs.
- It was commercially deployed by the US Postal Service for automated reading of zip codes on mail.
- It proved the effectiveness of parameter sharing and local receptive fields.
- LeNet-5 inspired all subsequent CNN architectures including AlexNet, VGGNet, GoogLeNet, and ResNet.

Q5. Explain the architecture of AlexNet. Highlight the improvements it introduced over LeNet, such as ReLU activation, dropout, and GPU usage. Discuss its impact on the ImageNet competition.

Answer:

AlexNet was designed by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton in 2012. It won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2012 with a top-5 error rate of 15.3%, outperforming the second-best entry by a margin of 10.8 percentage points. This victory is widely regarded as the event that ignited the modern deep learning revolution.

Architecture of AlexNet

AlexNet contains 8 learnable layers — 5 convolutional layers and 3 fully connected layers — with approximately 60 million parameters:

- Input: 227 × 227 × 3 color image.
- Conv1: 96 filters of size 11×11, stride 4 → Output: 55×55×96. Followed by ReLU and Max Pooling (3×3, stride 2).
- Conv2: 256 filters of size 5×5, stride 1, padding 2 → Output: 27×27×256. ReLU + Max Pooling.
- Conv3: 384 filters of size 3×3, stride 1, padding 1. ReLU.
- Conv4: 384 filters of size 3×3, stride 1, padding 1. ReLU.
- Conv5: 256 filters of size 3×3, stride 1, padding 1. ReLU + Max Pooling.
- FC6: 4096 neurons. ReLU + Dropout (50%).
- FC7: 4096 neurons. ReLU + Dropout (50%).
- FC8: 1000 neurons (ImageNet classes). Softmax output.

Key Improvements Over LeNet

- **ReLU Activation:** AlexNet used ReLU (f(x) = max(0, x)) instead of sigmoid/tanh. ReLU is computationally cheaper and avoids the vanishing gradient problem, enabling faster and deeper training.
- **Dropout Regularization:** During training, dropout randomly sets 50% of neuron outputs to zero in fully connected layers. This prevents co-adaptation of neurons and acts as a powerful regularization technique to reduce overfitting.
- **GPU Training:** AlexNet was the first major CNN trained on GPUs (specifically, two NVIDIA GTX 580 GPUs with 3GB memory each). The model was split across two GPUs, enabling parallel computation and significantly reducing training time.
- **Data Augmentation:** Techniques such as random cropping, horizontal flipping, and PCAbased color augmentation were used to artificially increase the training set size and improve generalization.
- **Overlapping Pooling:** AlexNet used overlapping max pooling (3×3 window, stride 2), which improved performance compared to non-overlapping pooling.

Impact on ImageNet

AlexNet's dominant performance at ILSVRC 2012 demonstrated that deep CNNs could achieve superhuman-like accuracy on large-scale image recognition tasks. It catalyzed the shift from hand-crafted features to learned representations and launched the era of deep learning in computer vision, NLP, and beyond.

Q6. Compare LeNet and AlexNet in terms of architecture depth, dataset used, computational requirements, and performance. What advancements made AlexNet significantly more powerful?

Answer:

LeNet-5 and AlexNet represent two milestones in the history of CNNs, separated by over a decade. The following table provides a detailed comparison:

Comparison Table

Feature	LeNet-5 (1998)	AlexNet (2012)
Total Layers	7 (2 Conv + 2 Pool + 3 FC)	8 (5 Conv + 3 FC) + Pooling
Input Size	32 × 32 (grayscale)	227 × 227 × 3 (color)
Dataset	MNIST (60,000 images, 10 classes)	ImageNet (1.2M images, 1000 classes)
Parameters	~60,000	~60 million
Activation Function	Tanh / Sigmoid	ReLU
Pooling	Average Pooling	Overlapping Max Pooling
Regularization	None	Dropout (50%)
Training Hardware	CPU	2 × NVIDIA GTX 580 GPUs
Data Augmentation	None	Yes (cropping, flipping, PCA jitter)
Normalization	None	Local Response Normalization (LRN)
Performance	Excellent on MNIST	Won ILSVRC 2012 (top-5: 15.3%)

Key Advancements in AlexNet

- **Deeper Architecture:** AlexNet is significantly deeper and wider, with 5 convolutional layers and 3 fully connected layers, allowing it to learn more complex hierarchical features.
- **ReLU Activation:** ReLU solved the vanishing gradient problem and allowed training of much deeper networks with faster convergence.
- **Dropout:** Dropout prevented overfitting on the large ImageNet dataset, a critical improvement.
- **GPU Computing:** Training on GPUs made it feasible to train networks with 60 million parameters on over 1 million images.
- **Larger Dataset:** The scale and diversity of ImageNet provided richer feature learning compared to MNIST.
- **Data Augmentation:** Techniques like random cropping, flipping, and color augmentation improved generalization.

In conclusion, while LeNet-5 laid the foundational principles of CNN architecture, AlexNet scaled these ideas with modern hardware, larger data, and algorithmic innovations, proving that deep learning could solve real-world visual recognition tasks at scale.

Q7. What is CNN visualization? Explain the need for visualization techniques in understanding deep learning models. Discuss feature map visualization with suitable examples.

Answer:

What is CNN Visualization?

CNN visualization refers to a set of techniques used to understand and interpret what a Convolutional Neural Network has learned internally. Since CNNs are often considered "black boxes," visualization helps researchers and practitioners gain insights into the internal representations, feature detections, and decision-making processes of the network.

Need for Visualization

- **Interpretability:** CNNs can have millions of parameters. Visualization helps understand what features drive the network's decisions.
- **Debugging:** If a model makes incorrect predictions, visualizing intermediate layers can help identify where the network's understanding breaks down.
- **Trust and Accountability:** In domains like healthcare, autonomous driving, and security, understanding why a model made a decision is critical for trust and regulatory compliance.
- **Bias Detection:** Visualization can reveal if a model has learned meaningful features or is relying on spurious correlations (e.g., recognizing a watermark instead of an object).
- **Architecture Design:** By understanding what each layer captures, researchers can design better architectures.

Feature Map Visualization

Feature map visualization involves displaying the activations (output) of intermediate convolutional layers when an input image is passed through the network. Each feature map shows what a specific filter has detected.

Example — Visualizing layers when a cat image is input:

- **Layer 1 (Early):** Feature maps show responses to basic edges (horizontal, vertical, diagonal), gradients, and color blobs. These are generic, low-level features.
- **Layer 2–3 (Middle):** Feature maps capture textures, patterns, corners, and combinations of edges. For a cat, these might show fur texture patterns or whisker-like curves.
- **Layer 4–5 (Deeper):** Feature maps represent high-level semantic features like eyes, ears, nose, or the overall face shape. These are class-specific features.

This progression from low-level to high-level features demonstrates the hierarchical nature of CNNs. Feature map visualization can be done by simply extracting and plotting the activations at any given layer using frameworks like PyTorch or TensorFlow.

Q8. Explain Guided Backpropagation as a CNN visualization technique. How does it differ from standard backpropagation? Discuss its role in interpreting learned features.

Answer:

What is Guided Backpropagation?

Guided Backpropagation is a visualization technique proposed by Springenberg et al. (2015) that produces a saliency map showing which input pixels most strongly influence a particular neuron's activation. It modifies the standard backpropagation process to generate cleaner and more interpretable visualizations.

Standard Backpropagation vs. Guided Backpropagation

In standard backpropagation, during the backward pass through a ReLU activation, the gradient is passed through if the input to the ReLU was positive (i.e., the gate was "open" during the forward pass), and blocked if it was negative:

Standard ReLU Backward:

If forward input > 0: pass gradient

If forward input ≤ 0: block gradient (set to 0)

In guided backpropagation, an additional constraint is applied: the gradient is passed through only if BOTH the forward input was positive AND the incoming gradient from the upper layer is also positive:

Guided Backpropagation ReLU Backward:

If forward input > 0 AND incoming gradient > 0: pass gradient

Otherwise: block gradient (set to 0)

This means that only the gradients corresponding to neurons that had a positive influence in both the forward and backward directions are propagated. Negative gradients (which represent suppressive effects) are masked out.

Comparison Table

Aspect	Standard Backpropagation	Guided Backpropagation
Gradient through ReLU	Passed if input > 0	Passed if input > 0 AND gradient > 0
Negative gradients	Propagated	Masked (set to zero)
Visualization quality	Noisy, less interpretable	Clean, sharp, interpretable
Focus	All influences	Only positive influences

Role in Interpreting Learned Features

- Guided backpropagation produces high-resolution visualizations that highlight exactly which parts of the input image are responsible for activating a specific neuron or class.
- It reveals the specific patterns (edges, textures, shapes) that each neuron has learned to detect.
- It helps verify that the network is focusing on semantically meaningful regions (e.g., the face of a dog for class "dog") rather than irrelevant background features.
- Combined with other techniques like Grad-CAM, guided backpropagation provides comprehensive model interpretability.

Q9.

Describe the concepts of Deep Dream and Neural Style Transfer (Deep Art).

Answer:

Deep Dream

Deep Dream is a visualization technique developed by Google (Alexander Mordvintsev et al., 2015) that creates surreal, psychedelic images by amplifying the patterns detected by a neural network. It works by the following process:

- **Step 1:** Pass an input image through a pre-trained CNN (e.g., GoogLeNet/Inception).
- **Step 2:** Select a specific layer of the network whose activations you want to maximize.
- **Step 3:** Compute the gradient of that layer's activation with respect to the input image.
- **Step 4:** Modify the input image by adding these gradients (gradient ascent), which enhances the features the network already detects.
- **Step 5:** Repeat the process iteratively, often at multiple scales (octaves), to produce increasingly vivid and dreamlike images.

The result is an image where patterns such as eyes, faces, animals, and fractal structures appear to emerge from noise or ordinary photographs. Selecting different layers produces different levels of abstraction — early layers create edge-like patterns, while deeper layers produce complex object-like hallucinations.

Neural Style Transfer (Deep Art)

Neural Style Transfer (NST), proposed by Gatys et al. (2015), is a technique that combines the content of one image with the artistic style of another image to create a new, stylized image. It leverages a pre-trained CNN, typically VGG-19.

The process works as follows:

- Content Representation — extracted from deeper layers (e.g., conv4_2 of VGG-19), which capture the spatial structure and arrangement of objects in the image.

- Style Representation — extracted from multiple layers using Gram matrices, which capture the correlations between different feature maps. These correlations represent textures, colors, and artistic patterns.
- Generated Image — starting from random noise (or a copy of the content image), the algorithm optimizes the pixel values to simultaneously minimize the content loss (difference from content image) and the style loss (difference from style image).

For example, combining a photograph of a cityscape (content) with Van Gogh's "Starry Night" (style) produces a cityscape painted in Van Gogh's swirling, vibrant style.

Q10.

Explain the objective function used in Deep Dream.

Answer:

The core idea behind Deep Dream is to modify an input image such that the activations of a chosen layer in the neural network are maximized. This is achieved through gradient ascent on the input image.

Objective Function

The objective function in Deep Dream is defined as:

$$L = \Sigma_{ij} (a_{ij}^{l})^2$$

Where:

a_{ij}^{l} = activation of the i-th feature map at spatial position j in layer l

L = the objective to be maximized

The goal is to maximize the L2 norm (sum of squares) of the activations at the selected layer. This forces the network to amplify whatever patterns it already detects in the image.

Optimization Process

Unlike training (where we minimize a loss w.r.t. weights), in Deep Dream we perform gradient ascent on the input image while keeping network weights frozen:

1. Forward pass: Compute activations at the chosen layer l.
2. Compute gradient: Calculate $\partial L/\partial x$ (gradient of the objective w.r.t. the input image x). 3. Update input: $x \leftarrow x + \eta \cdot (\partial L/\partial x)$ where η is the learning rate / step size. 4. Repeat steps 1–3 for multiple iterations.

Multi-Scale Processing (Octaves)

To produce visually richer results, Deep Dream uses multi-scale processing:

• The input image is progressively scaled up through several "octaves."

• At each scale, the gradient ascent optimization is applied.

• Details generated at smaller scales are preserved and refined at larger scales.

This produces fractal-like, self-similar patterns that give Deep Dream images their distinctive surreal appearance.

Layer Selection

The choice of layer significantly affects the result:

• Early layers (e.g., conv1, conv2): Produce simple edge and texture patterns.

• Middle layers: Generate more complex motifs like spirals and geometric shapes.

• Deep layers (e.g., inception_4a): Produce recognizable objects such as eyes, faces, and animals.
